

Beyond the Scope: Security Testing of Permission Management in Team Workspaces

Liuhuo Wan^{ID}, Chuan Yan, Zihan Wang^{ID}, Mark Huasong Meng^{ID}, Kailong Wang^{ID}, Haoyu Wang^{ID}, Guangdong Bai^{ID}, *Member, IEEE*, and Jin Song Dong^{ID}

Abstract—Nowadays team workspaces are widely adopted for multi-user collaboration and digital resource management. To broaden their applicability in real-world scenarios, mainstream team workspaces, such as Google Workspace and Microsoft OneDrive, support the integration of third-party applications (referred to as *add-ons*). These add-ons significantly extend the functionality of team workspaces. Combined with the powerful multi-user collaboration capabilities, this extensibility positions team workspaces a central hub for managing shared resources and enforcing access control. However, unlike in single-user environments where permission management is more straightforward, the collaborative nature of team workspaces may allow add-ons to bypass administrator-enforced permission isolation, potentially introducing new security risks. This article aims to investigate the permission management of add-ons. Our study starts with an in-depth analysis of the enforced access control mechanism inherent in this ecosystem. We identify three potential security risks that can be exploited to cause permission escalation. Based on this knowledge, we develop a tool named *TAI* to automatically test all possible permission-related interactions. We apply *TAI* to systematically understand the landscape of permission escalation risks in the current ecosystem. Our evaluation reveals that permission escalation vulnerabilities are widespread in this ecosystem, with 41 interactions identified as problematic. Our findings should raise an alert to both the team workspaces platforms and third-party developers.

Index Terms—Access control, industry applications, security.

I. INTRODUCTION

TEAM workspaces platforms serve as unified collaboration hubs that streamline business operations and day-to-day workflows. For example, users can manage daily tasks in a shared spreadsheet, and other collaborators can access it directly with roles such as viewer, commenter, or editor. Prominent examples of team workspaces include Google Workspace [1]

Received 9 August 2025; revised 26 May 2026; accepted 14 July 2026. Date of publication 21 July 2026; date of current version 1 September 2026. This work was supported by the Ministry of Education, Singapore under its Academic Research Fund Tier 3 under Grant MOET32020-0003. (*Corresponding author: Guangdong Bai.*)

Liuhuo Wan and Zihan Wang are with the University of Queensland, Brisbane, QLD 4072, Australia.

Chuan Yan and Guangdong Bai are with the City University of Hong Kong, Kowloon, Hong Kong (e-mail: g.bai@cityu.edu.hk).

Mark Huasong Meng is with University College Dublin, Dublin 4 Belfield, Ireland.

Kailong Wang and Haoyu Wang are with the Huazhong University of Science and Technology, Wuhan 430074, China.

Jin Song Dong is with the National University of Singapore, Singapore 119077.

Digital Object Identifier 10.1109/TDSC.2026.3715679

and Microsoft OneDrive [2]. The demand for online and remote collaboration, particularly accelerated by the effects of the pandemic, has propelled the widespread adoption of these team workspaces. Google Workspace, for example, reports over three billion users [3] and is adopted by nine million companies as their business solution. Team workspaces can be further enhanced by integrating third-party applications (named *add-ons*) to supplement advanced features. They not only provide extended functionality but can also incorporate external services such as Confluence, Evernote, GPT, Zoom, Dropbox, and We-bex. While such integration enhances the user experience, it also introduces complexities in permission management when accessing sensitive resources hosted by team workspaces.

An add-on accesses resources managed by team workspaces through host APIs, for tasks such as viewing documentation or replying to an email. These APIs are regulated by different permissions. Each add-on is subject to a two-level permission model that combines *OAuth permission scope checking* and *user role checking*. In detail, during installation, the add-on requests OAuth tokens with permission scope, which are used to interact with resources managed by team workspaces. For example, an add-on might request the OAuth permission to ‘*view and edit all Google Documents*’ from the user. However, some resources accessed by the add-on during runtime may not be owned by the user. If the user is assigned the viewer role and has no permission to edit the resource, the add-on installed by that user is similarly prohibited from modifying the resource. Notably, this two-level permission model delays authorization checks to runtime, where they are enforced by the team workspace. However, in practice, team workspaces employ two separate permission-checking systems for users and add-ons. Since these systems operate independently, inconsistencies may arise between the permissions granted to a user and the actions an add-on can perform on the user’s behalf. This issue is particularly evident under customized access-control settings defined by workspace owners. Thus, the permission management for add-on inevitably introduces new attack surfaces, posing risks to security-critical collaboration. Despite its significance, this issue has not been thoroughly studied before.

Shadow of security risks: Team workspaces support multi-user collaboration through role-based permission management, where an owner retains full privileges and assigns partial privileges to other roles as shown in Fig. 1. It is expected that a user role does not exceed the permission scopes assigned by the owner. Similarly, add-ons installed by a specific user role (e.g.,

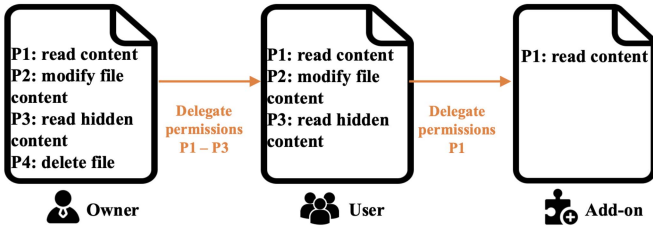


Fig. 1. Permission delegation demonstration.

viewer) should adhere to the permission scopes delegated by this user. This raises a critical question: *does permission management remain consistent across different users and add-ons?* For example, can an unprivileged member install an add-on to access resources they are not entitled to, such as hidden rows or columns in a protected spreadsheet, and even modify them? It is important to note that the installation of add-ons in private workspaces (e.g., personal workspace) typically requires no vetting process [4]. In such scenarios, if permission management is not well-established, an add-on can become a vehicle for permission escalation.

Despite the severe security risks brought about by add-ons, limited efforts [4], [5], [6] have been devoted to understanding the security issues they pose due to the opaque nature of both team workspaces and add-ons. Furthermore, the complexity of team workspaces' features, including multi-user roles, complicates the analysis and testing of these security risks. Existing efforts [4], [5], [6] predominantly rely on manual interactions and inspections. Consequently, **there is a need for automated tools to test and analyze permission management considering the integration of add-ons.**

To comprehensively evaluate the underlying permission escalation problem caused by the add-ons, we systematically test and analyze the add-ons' capability to utilize host APIs for achieving permission escalation. Specifically, we develop a tool named *TAI* to automatically generate test cases and evaluate the add-ons' capability under different permission scopes. *TAI* identifies 41 risky host APIs that add-ons can exploit, leading to permission escalation. These APIs affect prominent services like Spreadsheets, Documents, Drive Files, and Forms. They allow unauthorized operations on resources, including viewing hidden data protected by the admin, umodifying specific data groups without authorization, and interfering with existing members of shared resources.

Contributions: The contributions of this work are summarized as follows:

- *An in-depth understanding of the permission system:* We conduct a detailed analysis and reveal the complex permission control mechanisms within team workspaces: the two-level permission checking for add-ons. This enables us to understand the precise mechanisms that govern data access and permission requests. Based on this established knowledge, we can identify and formally define the permission escalation vulnerabilities in the current system.
- *A systematic tool for the permission escalation testing:* To comprehensively detect problematic usage of add-ons

host APIs that could lead to permission escalation, we have devised and implemented an automated tool called *TAI*. *TAI* is built on a pipeline that includes permission categorization, test case generation, and user-role-based testing. Using *TAI*, we can detect discrepancies during the permission-checking process. This systematic scanning approach enables us to identify all risky API usages.

- *The landscape of permission escalation issues:* Our research has uncovered a large number of problematic APIs. Through detailed discussions supported by concrete case studies, we have identified potential attack scenarios that could arise from problematic API usage. Furthermore, we have proposed countermeasures to mitigate the permission escalation issues for team workspaces and end-users.

Ethics and Disclosure: All our experiments are conducted using test accounts. The workspace is controlled, with the authors as the only members. The malicious add-ons designed are only installed in the controlled team workspaces and access limited resources. We did not distribute these malicious add-ons into other team workspaces or the public marketplace. All our attacks would not affect users and resources other than the authors' testing accounts. We ethically disclosed our findings to team workspaces and received acknowledgment.

II. PROBLEM FORMULATION

A. Background and the Permission System

Before the launch of team workspaces, collaboration in organizations often relied on a combination of tools that were not as seamlessly integrated or cloud-based as modern solutions. Different applications often require manual syncing and are not always designed for real-time, cloud-based collaboration. For example, Dropbox for file storage and Slack for messaging operate in separate environments, requiring users to juggle multiple applications and interfaces. This fragmentation makes it challenging to maintain a cohesive workflow, as data has to be manually transferred, and users have to manage different accounts and logins. Team workspaces bring a more integrated, cloud-native approach by combining email, chat, file storage, document editing, and scheduling tools into a single, unified platform. This integration allows for real-time collaboration on documents, seamless communication via email, and centralized file storage and sharing—all accessible through a web browser without downloading separate desktop applications. As a result, team workspaces significantly simplify workflows, reduce the reliance on disparate tools and create a more streamlined, efficient collaboration environment for many organizations.

Sharing is one of the most powerful features supported by team workspaces. Users can seamlessly share their resources and engage in real-time collaborative file editing, eliminating the need for redundant resource distribution.

Users: There are four types of collaborators supported in the current ecosystem of team workspaces: owner, editor, commenter and viewer. The owner has all privileges related to shared resources, while other roles have varying permission scopes, as suggested by their names. The owner can assign different roles to the collaborators.

TABLE I
DEFINITION OF SYMBOLS

#	Symbol	Name	Definition
Basic symbol			
1	$\mathcal{S}$	Subject	Owner, Editor, Commenter, Viewer, Unauthorized User and Add-on
2	$\mathcal{A}$	Action	Create, View, Comment, Modify and Delete
3	$\mathcal{O}$	Object	Calendar, Document, Drive, Gmail, Forms, Spreadsheet, Slides
4	$\mathcal{P}$	Permission	$\mathcal{P} = \mathcal{S} \times \mathcal{A} \times \mathcal{O}$
5	$\mathbb{P}_{\text{user}}$	User permission	$\{p_u \mid p_u : (s_u, a, o), s_u \in \mathcal{S} \setminus \{\text{add-on}\}, a \in \mathcal{A}, o \in \mathcal{O}\}$
6	$\mathbb{P}_{\text{addon}}$	Add-on permission	$\{p_a \mid p_a : (\text{add-on}, a, o), a \in \mathcal{A}, o \in \mathcal{O}\}$
7	$\mathcal{P}_{\text{con}}$	Permission consistency	$\forall U \subseteq \mathcal{S}, \mathbb{P}_{\text{addon}}^{(U)} \subseteq (\mathbb{P}_{\text{user}} \wedge \text{Auth})$ $\mathbb{P}_{\text{addon}} \subseteq \mathbb{P}_{\text{authen}} \subseteq \mathbb{P}_{\text{user}} \subseteq \mathbb{P}_{\text{owner}}$
Permission escalation			
8	E_1	OAuth permission escalation	$\mathbb{P}_{\text{addon}} \not\subseteq \mathbb{P}_{\text{authen}}$
9	E_2	User role permission escalation	$(\mathbb{P}_{\text{addon}} \subseteq \mathbb{P}_{\text{authen}}) \wedge (\mathbb{P}_{\text{addon}} \not\subseteq \mathbb{P}_{\text{user}})$
10	E_3	Sharing permission escalation	$\text{Addn.modify}(\mathbb{C}) \rightarrow \mathbb{C}'$, where $\mathbb{C} \neq \mathbb{C}'$

Add-ons: Users can integrate add-ons into their team workspaces. Beyond interactions with legitimate users, add-ons can access resources through user delegation, as shown in Fig. 1. With permission scopes granted by users, add-ons can access and manipulate resources stored within the user's workspace. These resources fall into two categories: those for whom this user is the owner and those for whom this user is a collaborator (e.g., viewer or editor).

B. Permission System

Objects: There is a list of resources supported by the team workspaces, such as documents, spreadsheets, presentations, and forms. Each resource exhibits a hierarchical structure composed of finer-grained components. For example, a spreadsheet consists of rows and each row contains multiple cells. Every cell may carry its own set of attributes, such as content value, background color, and formatting. We refer to all of these components, at any level of granularity, as objects.

Subjects: In total, team workspaces define five subjects as shown in Table I: the owner, viewer, commenter, editor and add-ons that can access predefined objects. The unauthorized user is not allowed to access predefined objects.

Actions: The team workspaces support five types of actions as listed in the Table I.

Permissions: We represent each permission in team workspaces using a tuple (*subject*, *action*, *object*) as shown in Table I, which indicates who (the *subject*) can perform what (the *action*) on which resource (the *object*). The detailed examples is shown in the Table I. For example, the permission (*add-on*, *delete*, *Document*) indicates the capability of an add-on to delete the entire *Document*.

Team workspaces enforce a two-level access control model for add-ons, as illustrated in Fig. 2. When the add-on utilizes host APIs like `editText()` to access the installer's resources, team workspaces verify the authorized **OAuth** permission scopes granted by the installer and the installer's **role** as a collaborator associated with the resource.

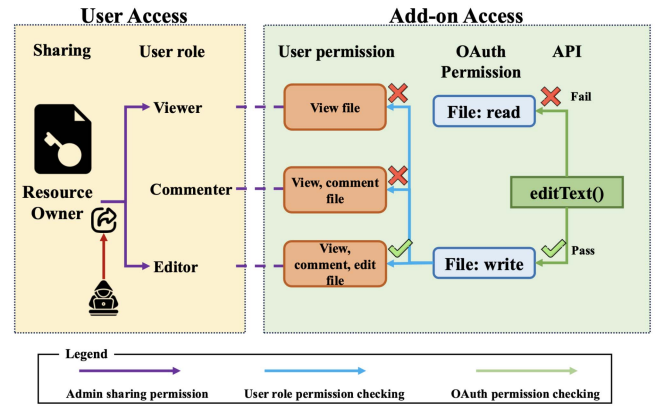


Fig. 2. Access control mechanisms in the current ecosystem.

a) Level 1: OAuth permission scope checking : As illustrated by the green lines in Fig. 2, when the add-on tries to execute the API `editText()`, team workspaces first check the OAuth permission scopes already authorized by its installer. If the add-on has been granted `file:write` permission, it is allowed to write to the file and successfully passes the first-level permission check. In contrast, if the add-on is only authorized with `file:read` permission, it will fail to execute the `editText()` API due to insufficient permissions.

b) Level 2: User role permission checking : After successfully completing the first-level permission check, team workspaces then evaluate the installer's role on the resource. As shown by the blue lines in Fig. 2, for the `editText()` API, even if the installer has granted the add-on with `file:write` permission, the second-level permission check will fail if the installer's role for the resource is set to *viewer* or *commenter* without editing privileges. As a result, team workspaces return an exception.

Based on this two-level permission checking, we can conclude that the add-on is constrained by both granted permission scope and installer's roles: $\mathbb{P}_{\text{addon}} \subseteq \mathbb{P}_{\text{authen}} \subseteq \mathbb{P}_{\text{user}}$. For example, when a user assumes the role of a viewer, both the actual user and the installed add-on are restricted from editing this resource.

C. Permission Escalation Definitions

Add-ons must adhere to the permission system enforced by team workspaces. If team workspaces fail to enforce these permission checks, add-ons will achieve permission escalation, as indicated by the red line or cross mark in Fig. 2. Our work focuses on **permission escalation** issues that arise from the add-on's use of host APIs. We formalize our attack model as follows.

Definition II.1 (Permission Control Consistency): As shown in Table I, we let $\mathcal{O}$ represent the set of object fields (e.g., spreadsheet, document); $\mathcal{A}$ denotes a set of operations that can be applied to the object; $\mathcal{S}$ represents the subjects.

Permission representation: The user permission $\mathbb{P}_{\text{user}}$ and the add-on permission $\mathbb{P}_{\text{addon}}$ are defined as line #5 and #6 in Table I.

Permission consistency representation: The two-level permission checking model can be represented as #7 in Table I. For each add-on installed by user U , the add-on permission scope is confined by user role U and cannot exceed the permission scope authorized by U , represented as Auth .

Definition II.2 (Permission Escalation). We delineate three types of permission escalation scenarios. The first two scenarios stem from two-level permission checking of add-ons. The final scenario arises from resource-sharing permission checks. These three attacks are demonstrated as red crosses or red lines, as shown in Fig. 2. Each of these scenarios can lead to permission escalation [7], [8].

OAuth permission escalation (E_1): When the add-on installed in the user's team workspaces can exceed the authorized permission scopes, this violates the first level permission check and results in permission escalation. For example, if the add-on is authorized only with the `file:read` permission but can still use `editText()` to write to the file (green arrows in Fig. 2), this results in a permission escalation.

User role permission escalation (E_2): When an add-on installed in the user's team workspace can perform actions that the user is not permitted to take, it clearly violates the second-level permission check. For example, we find that:

$$\left\{ \exists a \in \mathcal{A}, \exists o \in \mathcal{O}(\text{add-on}, a, o) \in \mathbb{P}_{\text{addon}}^{(U)} \wedge (s_u, a, o) \notin \mathbb{P}_{\text{viewer}} \right\}$$

The $\mathbb{P}_{\text{addon}}^{(U)}$ refers to an add-on installed by a user with role U . The add-on can read some specific content of the resource, whereas its installer cannot (blue arrows in Fig. 2).

Sharing permission escalation (E_3): Besides permission checking when a user is assigned a role, we also consider permission sharing. As shown in Fig. 2, sharing permissions are managed by the administrator, who can grant resource access to collaborators. However, if a user's assigned role is stealthily modified by add-ons, altering the sharing topology configuration $\mathbb{C}$ (purple lines in Fig. 2) without administrator action, this also constitutes a permission escalation. We define this attack scenario as #10 in Table I.

Overview across representative platforms: We present an overview of the two most popular and representative team workspaces, Google Workspace and OneDrive. As shown in Table II, all of these platforms adopt a two-level permission check system. They enable user-sharing features with four types

TABLE II
THE IMPLEMENTATION OF TEAM WORKSPACES AND THEIR POTENTIAL PERMISSION ESCALATION

Platform	Two-level permission system	E_1	E_2	E_3
Google Workspace	✓	○	○	○
Microsoft OneDrive	✓	○	○	●

A check mark (✓) indicates that the platform adopts the two-level permission system identified in our study. A red circle (○) signifies potential vulnerability to permission escalation. A black circle (●) signifies no vulnerability to permission escalation.

of user roles: owner, viewer, commenter, and editor. Both platforms are vulnerable to the first two permission escalation issues. However, OneDrive is robust against E_3 because it prohibits the capability of add-ons to modify user roles.

III. OVERVIEW AND CHALLENGES

Detecting the predefined three scenarios requires a thorough understanding of the permission scope for each role in the current team workspaces. However, the existing documentation on permission scope is incomplete. While previous work [4] provides partial insights into permission scopes from the user side, it does not address the scope from the add-on perspective. In this study, we systematically investigate the relevant permission scopes from different perspectives: real users and delegated add-ons.

a) Real-users : To understand the proper permission scopes from user side, we establish five test accounts, each representing one of the five user roles in team workspaces. Subsequently, we employ a manual approach to comprehensively explore real-user roles and their associated permission scopes by assuming different roles. The exploration of the user permission scope was a one-time effort and was conducted manually. We investigate the capabilities of each user role with respect to the fundamental permission groups, which will be detailed in Section IV-A.

b) Add-ons : Unlike assessing the permission scope for real users, which can be evaluated through manual review by assuming different user roles, the permission check of the add-on involves the invocation of a large number of host APIs. The security implications of these APIs cannot be effectively analyzed without the assistance of an automated tool. Therefore, we have designed an automation tool called *TAI* that is capable of analyzing APIs, generating test cases, and automatically invoking APIs based on the official documentation [9] provided by team workspaces.

Our framework, called *TAI*, is a Team workspace Add-on API testing tool that aims to test each API based on the official documentation provided by team workspaces. It first categorizes each API to the corresponding permission group, generates the correct test cases for each API, then executes the API under different user roles and record the execution status. Finally, we check whether the three predefined permission escalation scenarios happened. For example, if a document editing API is tested within a viewer workspace and still executes successfully,

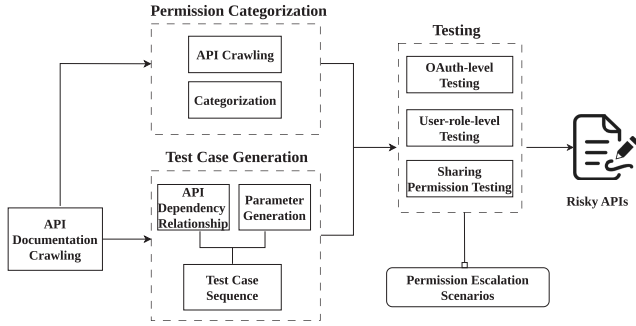


Fig. 3. Workflow of TAI.

it indicates permission escalation E_2 . We identified three challenges when implementing the TAI due to features specific to team workspaces.

- **Challenge #1. API hierarchy:** There is a hierarchical structure among host APIs (shown in Fig. 4), which is different from other web-based applications that use a flat HTTP request API for resource access [10], [11]. Existing work [4], [10], [11], [12] on OAuth-based web applications like Slack and GitHub cannot handle this API hierarchy. In particular, host APIs have interdependent relationships, where certain APIs must be invoked before others. For example, to execute an API that edits document content, the API call chain that returns the current active document, `DocumentApp.getActiveDoc()`, must be invoked first. Therefore, the primary challenge lies in precisely capturing the hierarchical structure of host APIs.
- **Challenge #2. Parameter generation:** Certain APIs require context-dependent parameters for successful execution. For instance, the API `DriveApp.getFolderById(ID)` needs a valid folder ID. To properly invoke this API, our TAI must be able to supply a valid folder ID as input.
- **Challenge #3. Efficiency:** Team workspaces encompass a huge number of APIs, and testing every single one is neither efficient nor imperative for our objectives. Our primary goal is to identify permission escalation, necessitating the timely filtration of unnecessary APIs. For example, if a viewer lacks the capability to execute the `Document.addFooter()` API, subsequently rendering all methods reliant on the `addFooter()` meaningless for testing (e.g., `Document.addFooter().setText('footer')`), these cases should be excluded. Consequently, TAI is specifically designed to address this challenge, ensuring the timely filtering of unnecessary test cases.

IV. DESIGN OF TAI

As shown in Fig. 3, TAI consists of three major components: permission extraction (Section IV-A), test case generation (Section IV-B) and testing (Section IV-C). Lastly, we detail the risky APIs in Section IV-D.

TAI is largely automated, with only minimal manual effort required for final verification. Specifically, TAI first crawls all

TABLE III
SUMMARY OF OBJECT COUNTS

Category	Calendar	Document	Drive	Form	Gmail	Spreadsheet	Slide	Total
Number	7	36	6	34	6	58	47	194

official APIs from the Google developer documentation [9]. It then constructs dependency relationships based on the documentation, generates test cases for all candidate APIs, and executes them accordingly. During execution, TAI records all responses and identifies potential permission escalations using the risky API identification method. The only components requiring manual effort are the initial setup of the add-on, its assignment to different user roles, and the final verification of risky APIs. The configuration and role distribution of the add-on is a one-time effort that takes approximately one hour to complete. The final verification step is also performed once and requires around two hours.

A. API Permission Extraction

We crawl all the API documentation including the API name, its parent class, description, parameters (parameter name, type, description, etc.), tutorial code snippets (if available), and the return type from the official developer website of team workspaces [9], as shown in Fig. 5. We use Selenium [13], the most widely used web crawling framework, to automate this crawling process. As shown in Fig. 4, team workspaces provide a standard hierarchy for objects at varying levels of granularity. For example, the objects `Body` and `Header` are fine-grained components of the `Document` object, while `Text` is a fine-grained component of the `Body` object. We adopt this hierarchy to derive the permission tuple for each API. The total number of objects supported by each application is shown in Table III.

API Crawling Frequency: We re-crawled the documentation one year later and observed that only a small fraction of the APIs have changed. Specifically, a limited number (79) of APIs were removed and a small number (21) of new APIs were added, while the vast majority remained unchanged. This suggests that the API documentation is relatively stable and does not change frequently in practice. Therefore, we recommend crawling all API information periodically (e.g., once per release cycle or on a monthly/quarterly basis, depending on the update frequency of the target platform) to ensure the dataset remains up to date.

We leverage the API's *name*, *description*, and the pre-extracted hierarchy to infer the intended semantic permission of each API. We aim to categorize the extracted permission groups into one of the following types: *(create, object)*, *(view, object)*, *(comment, object)*, *(modify, object)*, *(delete, object)* for later testing purpose. The object refers to the class on which the API is invoked. For example, `Document.getBody()` would be categorized as *(view, Document)*. In contrast, `Body.editAsText()` would be categorized as *(modify, Body)*. Notably, team workspaces may employ diverse verbs to convey the same operation (e.g., “get a file” and “read a file”, both signifying the

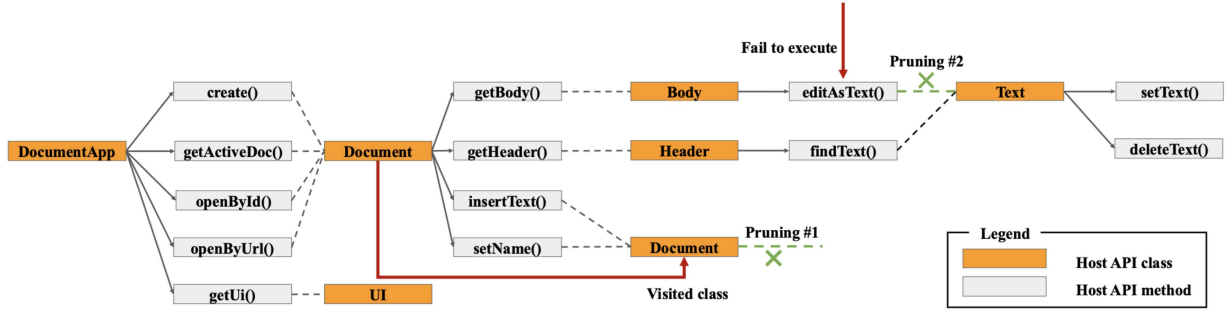


Fig. 4. An example of dependency tree of the APIs.

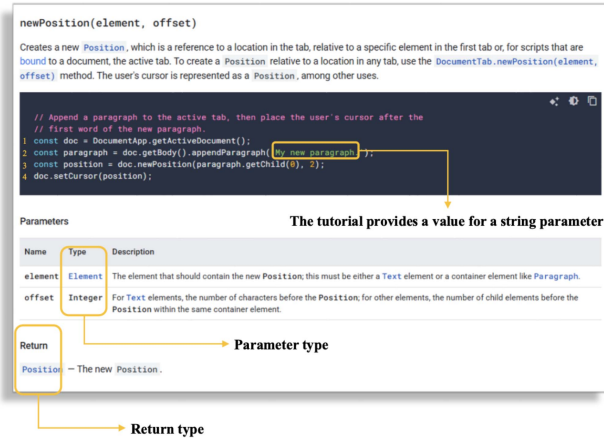


Fig. 5. Tutorial code snippets provided by team workspaces.

action of viewing a file). To address this, we adopt the powerful large language model: GPT capable of capturing the semantic meaning from diverse language descriptions. GPT is instructed to assign the correct permission group to each API. We use the gpt-4o-mini model [14] for this classification task. The example of the prompt we use is shown below. We set the temperature to 0 to make the LLM's outputs more deterministic. To evaluate repeatability, we run each query five times on our benchmark and measure the consistency of the results across runs. The performance remains consistent across different runs, achieving an average success rate of 99% across the benchmark, with failed cases accounting for only 1% of the benchmark. We also tested additional perturbed prompts on the same benchmark and evaluated their performance. We did not observe any significant deviation in the results.

You are an engineer who would like to utilize the following API. # Task Description I will provide you with the API name, its description and the context of the object hierarchy. Your task is to categorise the API into one of the operations: (create, view, comment, modify or delete). # Output Format [Output Format]

The OAuth permissions required for each API execution are documented in the team workspaces documentation.

Algorithm 1: Testing for API Calls.

Input G : Knowledge graph

Output C : candidates of testing APIs

```

1:  $T \leftarrow \emptyset, Visited \leftarrow \emptyset$ 
2:  $T \leftarrow [CalendarApp, DocumentApp, DriveApp, FormApp, GmailApp, SlidesApp, SpreadsheetApp]$ 
3: function Generate_test_cases()
4:   while  $T \neq \emptyset$  do
5:      $node = T.pop()$ 
6:     for all method  $api$  of  $node$  in  $G$  do
7:        $Class_{type} = API_{returnType}$ 
8:       if  $Class_{type}$  is not visited then  $\triangleright$  Pruning #1
9:          $T.add(Class_{type})$ 
10:         $C.add(dep : node, api)$ 
11:         $Visited \leftarrow Visited \cup Class_{type}$ 
12: function TestingC
13:   for each candidate  $c$  in  $C$  do
14:      $response = Execute\ c$ 
15:     if  $response \in errors$  then  $\triangleright$  Pruning #2
16:       filtering all APIs depend on the execution of  $c$ 
17:     if  $response \notin errors$  then
18:       record the  $response$ 

```

Therefore, the extracted API permissions can be precisely matched to the corresponding OAuth scopes. However, the permission scopes of different collaborators, represented as a tuple (*collaborator*, *a*, *o*), are unclear. To address this, two authors independently reviewed the extracted permissions. They used the roles of viewer, commenter, and editor to verify whether the extracted permissions were actually granted to the collaborators. In total, there are 194 objects and 5×194 permissions to be checked. The verification process took approximately eight hours to complete. In cases of disagreement, the two authors discussed the inconsistencies and reached a consensus through joint review.

B. Test Case Generation

1) *API Dependency Graph Construction*: We preserve the dependency between APIs (i.e., **Challenge #1**) and construct the dependency relationship based on three components: *classes*, all *methods* (APIs) of each class, and the *return type* of each

TABLE IV
EXAMPLE OF THE RETURNED CLASS TYPE OF DIFFERENT APIs

	API	Returned Type	TAI Strategy
Example 1	<code>Document.getFootnotes()</code>	<code>Footnote[]</code>	<code>Document.getFootnotes()[0]</code>
Example 2	<code>Document.getFooter()</code>	<code>FooterSection</code>	<code>Document.getFooter()</code>

method (can be another class or void). To be specific, we connect the *class* to its *methods* (bold arrow in Fig. 4) and *method* to its *return type* (dashed line in Fig. 4) to construct this dependency relationship. For example, in Fig. 4, `DocumentApp` has multiple methods: `create()`, `getActiveDoc()`, `openById()` and `openByUrl()`. These four methods return the same type of class `Document`. Furthermore, `Document` has multiple methods: `getBody()` that returns the body of `Document` and `insertText()` that inserts text into the `Document`.

2) *Parameter Dependency*: Certain APIs require valid input parameters that are dependent on the outputs or responses of other APIs (i.e., **Challenge #2**). For example, in Fig. 5, the execution of `doc.setCursor(position)` (line 4 in the code snippet) requires a valid value of `position`. We refer to such parameters as context-sensitive. We categorize the API parameters into three types: context-sensitive parameters with tutorial snippets provided by team workspaces, context-sensitive parameters without tutorial snippets, and non-context-sensitive parameters.

Tutorial available: We observe that team workspaces provide tutorial code snippets for APIs requiring context related inputs, as shown in Fig. 5. These code snippets are well-structured, context-sensitive, and generally do not require further modification. Consequently, we directly leverage these tutorial code snippets to capture the context parameter dependencies associated with such APIs. The only modification we apply to the tutorial code snippets is as follows: if the tutorial code snippets require string inputs to identify specific resources, we replace them with our customized string values, which will be detailed shortly.

Tutorial unavailable:

- *Unique class*: If team workspaces do not provide code snippets for context-sensitive parameters, we search the dependency relationship (Fig. 4) to obtain a valid parameter. Team workspaces specify the *type* of each required parameter in the official documentation. For each parameter with a unique class type, as illustrated in Fig. 4, we traverse the dependency graph to find the shortest path that leads to the required **class**. For example, `DocumentApp.getActiveDoc().getPosition()` returns a valid input for the `Position` parameter. If multiple paths of the same length exist, we randomly select one. If the retrieved path returns an array of the required **class**, we fetch the first element from the array, as shown in Table IV. Since our purpose is to test permission escalation rather than functionality, its actual value does not make a significant difference as long as the input is valid.
- *String constant*: Specific APIs like `DocumentApp.openByName("name")` require a valid string constant for the `name` parameter. Unique class types can be

TABLE V
EXAMPLES OF RETURNED DATA TYPES OF DIFFERENT APIs

Data Type	Enumeration
Integer	0, 1, 5, 10
Boolean	True, False
Integer pair	GPT-generated response

precisely mapped through graph traversal whereas basic strings cannot. To address such cases, we maintain an attribute table of the accessed resources. This table records all the runtime values of string parameters (e.g., id, url, name).

Non context sensitive: For remaining parameters that do not fall into the previous categories, such as `Integer`, `Boolean` or context-irrelevant `String` (line 3 in Fig. 5), we simply enumerate several possible values. We simply enumerate several possible values as shown in Table V. For `Integer`, we try values like 0, 1, and 10. For cases where multiple `Integer` inputs are required and have dependencies between them, we query the LLM model (gpt-4o-mini) to fetch valid inputs. For example, in the API `copyFormatToRange(gridId, column, columnEnd, row, rowEnd)`, all four parameters are integers. According to the documentation of team workspaces, `column` refers to “the first column of the target range”, and `columnEnd` refers to “the end column of the target range”. There is an implicit dependency where `columnEnd` should be larger than `column`. We leverage the LLM model, which is capable of handling such dependencies based on the documentation.

We additionally deployed a local LLaMA-based model (Llama 3.2) for the non-context-sensitive setting to improve the stability of TAI. To ensure a fair comparison, we used the same prompts and configurations as those used for the GPT-based models.

3) *Test Case Sequence*: The code listing illustrated in Fig. 5 demonstrates that the execution of the API `doc.setCursor(position)` (line 4) is contingent upon the successful execution of the call chain `doc = DocumentApp.getActiveDocument()` (line 1), which is **Challenge #1**. To comprehensively explore all potential APIs and generate the correct sequence, we employ a breadth-first search (BFS) strategy that starts from all APIs belonging to the root classes. During the exploration process, the return value of each API invocation is analyzed, and any newly discovered classes are added to the candidate queue for further testing. Meanwhile, we maintain the execution status throughout the testing process, including the classes that have already been explored and the APIs tested under each class. If an API returns an object whose class has already been tested in a

previous iteration (i.e., **Challenge #3**), we prune that branch to avoid redundant exploration (referred to as **Pruning #1**), since revisiting the same class would not provide additional coverage. As shown in Fig. 4, test case generation begins at the root classes *DocumentApp* and sequentially explores each method using BFS traversal. During traversal, when we encounter a previously visited class, we apply a pruning mechanism. For instance, the API `insertText()` in Fig. 4 leads to a class *Document* that has already been visited; we promptly prune this branch and discontinue generating test cases along this path. Algorithm 1 shows the details of the BFS traversal from lines 3 to 11, and we apply the **Pruning #1** in line 8.

Due to implicit dependencies in resource operations, some actions must occur before others. For example, a file must be created before it can be modified. Also, deletion needs to be tested as the final operation in a sequence. Therefore, our test case generation starts with APIs under the create permission scope. Next, it proceeds to view, comment, and modify operations. Finally, it concludes with the scope of the delete permissions. Similarly, for sharing permission, we adhere to such design: initiating the addition of collaborators first, followed by viewing, modifying, transferring ownership, or removing collaborators.

Completeness of test case generation: Our test cases are always generated starting from the root classes. According to the official Google API documentation [9], all APIs and classes supported by Google Workspace are reachable from the root classes. Therefore, our exploration is sound and complete with respect to the constructed API dependency graph. We adopt a breadth-first search (BFS) strategy for exhaustive exploration. During the process, newly discovered classes and their corresponding APIs are continuously added to a candidate queue for testing. We iteratively execute this process, expanding the exploration frontier whenever new classes or APIs are discovered. The procedure terminates when no new classes or APIs can be added to the queue, i.e., when a fixed point is reached where all reachable classes and APIs have been explored and no unseen elements remain for testing. This ensures that the generated test cases are sufficient in the sense that they fully cover all reachable API states under the documented dependency structure.

C. Risky API Testing

Due to the requirements of team workspaces, the test cases can only be tested within the add-on configuration portal, as shown in Fig. 6.

1) **Development of Add-Ons:** Due to the intimate nature of add-ons, we designed and built a testing add-on that adheres to team workspace practices. We then distributed this add-on across workspaces with various user roles for testing purposes. It is worth noting that the design and development of the add-on is a one-time effort, taking approximately one hour to complete [15]. The add-on configuration portal is shown in Fig. 6.

2) **Test Case Execution:** At the start of each testing cycle, we fetch the candidate test cases (to be tested) and update them into the project portal before execution. We use the Python library *pyautogui* [16] to simulate various hotkeys and simulate developer behavior. This allows us to paste the testing candidate into

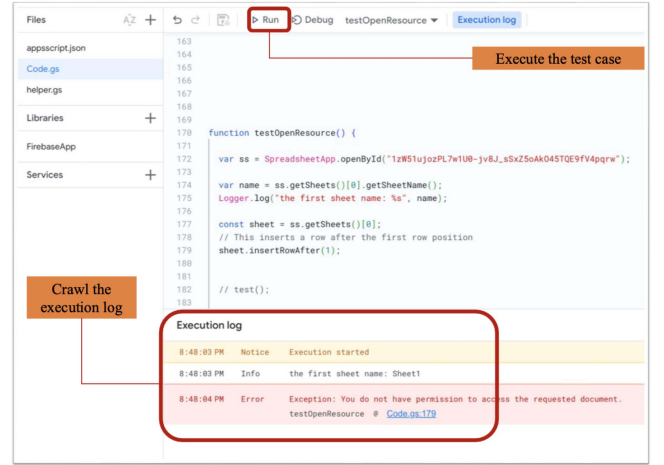


Fig. 6. The API testing phase.

the add-on configuration page (e.g., *Code.gs* in our scenario) using the `Ctrl + V` command. *Pyautogui* also simulates the save action (`Ctrl + S`) to save the changes to the add-on project. After updating the test cases, we automate the click actions on the *Run* buttons using *pyautogui* and recorded the execution log (bottom part of Fig. 6) of the test case as an indicator of its execution status.

The second pruning strategy is based on API dependencies. Specifically, some APIs can only be executed successfully if certain prerequisite APIs have already been executed successfully. Therefore, when an API fails to execute, all APIs that depend on its successful execution are considered infeasible and are consequently pruned from further testing. For example, the API `text.setText()` in Fig. 4 depends on obtaining a valid text object through `text = Body.editAsText()`. If `Body.editAsText()` cannot execute successfully during testing, the execution of `text.setText()` will also be blocked. In this case, we prune `text.setText()` along with other APIs (**Pruning #2**) that depend on `Body.editAsText()` to avoid unnecessary testing overhead caused by blocked executions.

OAuth-level testing: Based on the API permission categorization results, we utilize a progressive strategy when testing OAuth-level permission escalation (E_1). We begin by granting read-only permissions to the add-ons and then test whether they can successfully execute APIs that belong to the edit or delete groups. Next, we extend the permissions to include read and edit scopes to see if the add-ons can execute APIs that belong to the delete group. All testing was conducted using an owner account.

User-role-level testing: Google provides a comprehensive documentation regarding the user roles and how each user role is assigned. Under current design, user roles are strictly managed via the administrative invitation process. Specifically, the owner identity initializes the resources and assigns predefined roles (e.g., editor, viewer) to invited accounts. Crucially, the verification of these user role is enforced by Google's cloud environment during runtime rather than the client-side add-on itself. This represents a opaque security model in which the Google Workspace environment serves as the permission

enforcement point. In our testing, user roles were explicitly assigned by the Owner account during the invitation process to ensure that the add-on was executed under the intended privilege level.

We recognized that testing of sharing permission escalation could lead to state drift, where a user's role changes during execution. To maintain experimental integrity, we adopted a two-fold strategy: 1) Execution ordering: We prioritize the execution of E_1 and E_2 to leverage the initial, clean-state roles assigned by the owner. 2) Dynamic role mapping: For E_3 , where roles are subject to change, we maintain a dynamic lookup table to track the current status of each account. Rather than relying on the initial invitation settings, the framework queries this table to fetch the latest privilege value before every test case. This approach ensures that each test is conducted under the correct, real-time privilege level, thereby preserving the validity of our empirical results.

We created five user profiles for testing. To simulate different access-control scenarios, we created five separate Google Workspace accounts corresponding to five user roles defined in Table I: owner, unauthorized user, viewer, commenter, and editor. We used the owner account to create a set of template resources for each class to be tested, such as Google Docs, Google Sheets, Google Slides, and others. The owner account assigned different access permissions (i.e., editor or viewer) to the other accounts using their account ID. To avoid potential fingerprinting or cross-account correlation by Google, each user profile was created and tested on a separate computer. For each new round of testing, we utilized the same resource template to evaluate the execution of test cases. Each test case was tested under three different roles: viewer, commenter, and editor. Our goal is to test whether add-ons can bypass user-role-level permission checks. So the installer pre-authorizes all required permission scopes, ensuring that add-ons pass the OAuth-level check.

We monitor the execution status of each test case. If one API fails during testing (Fig. 4), we proceed to prune all subsequent API calls (**Challenge #3**) that depend on the successful execution of this API, referred to as **Pruning #2**. If the API passes the testing, we document the successful execution of the API and then continue evaluating subsequent API calls that rely on this one. The details of **Pruning #2** are demonstrated in lines 12–18 of Algorithm 1.

Sharing permission testing: We first identify a total of 45 APIs related to user role modification through simple keyword matching on API names containing terms such as *viewer*, *editor*, *owner*, and *sharing*. To systematically analyze their behavior, we leverage the test case generation approach to construct valid test cases. During testing, the add-on that perform the execution of test cases is installed by user roles other than the owner. We then execute these test cases and monitor the outcomes of the corresponding API calls. Furthermore, we examine whether such operations require explicit approval or trigger any notification to the owner.

3) *API Execution Result Monitor:* Although the response from a successfully invoked API may vary, the response from an invalid API invocation is more consistent. After conducting an

in-depth review of the team workspace developer documentation [9], we found that when the API fails to execute, the add-on will return a specific error message, such as “Exception: You do not have permission to access the requested document” and stop execution immediately. Whenever the execution returns error messages, the execution log outputs the type *Error*, as indicated in Fig. 6. We mark type *Error* as the indicator of a failed execution. On the other hand, API executions that do not return any error messages are deemed successfully executed and pass the testing process.

D. Risky API Identification

TAI evaluates whether the attack scenarios outlined in Section II-C are plausible by examining the runtime results of API invocations under different user roles. For example, if an API invocation that exceeds the user's permissions completes successfully without returning any errors, it suggests a potential security violation. However, verifying whether APIs designed to retrieve information return valid responses (as opposed to NULL or encrypted text) necessitates additional manual verification. We categorize the potentially risky APIs based on the criteria established in Section II-C.

Detection of E_1 : If the add-on is able to execute an API that falls outside the permission scope authorized by the user, it indicates a potential security issue or permission bypass: $p_a \in \mathbb{P}_{\text{addon}} \wedge p_a \notin \mathbb{P}_{\text{authen}}$. This will be detected as attack scenario E_1 .

Detection of E_2 : Similarly, if the add-on is able to execute an API that operates on resources but the user cannot: $\{p_a : (\text{add-on}, a, o) \mid p_u : (s_u, a, o), p_a \in \mathbb{P}_{\text{addon}} \wedge p_u \notin \mathbb{P}_{\text{user}}\}$. This will be detected as E_2 .

Detection of E_3 : If APIs that modify the current sharing configuration $\mathbb{C}$ can be executed successfully without requiring owner authorization or generating any notification, this indicates a lack of proper access control enforcement. In such cases, we conclude that a sharing permission escalation E_3 .

V. EVALUATION

Scope: Following the practice of previous study [4], [5], [17], we implement *TAI* and evaluate its performance on the most representative team workspaces, Google Workspace, which occupies around 73.04% market share [4].

A. Experiment Setup

We collected all official documentation, totaling 4,523 APIs across seven host applications, with an average of about 646 APIs per host application. To initiate automated API testing, we created multiple test accounts representing different user roles for collaboration. Our experiments were conducted on a set of computers, each dedicated to a different user role, to avoid potential device fingerprinting.

B. TAI Performance

The host application email has only one user role (owner) and does not support the collaboration feature, so we focus our

TABLE VI
TAI PERFORMANCE

Host App	# APIs	# Tested	Potential risky	Risky
Calendar	218	132	0	0
Document	846	47	8	6
Drive	152	142	25	14
Gmail	167	101	0	0
Forms	418	84	2	2
Spreadsheets	1784	624	15	15
Slides	938	46	4	4

TABLE VII
TEST CASE GENERATION

Category	# APIs	# Invalid [†]	Precision
No parameter required			
No parameter	518	0	100%
Tutorial available			
Tutorial	517	0	100%
Tutorial unavailable			
Path dependency	33	4	87.9%
String constant	70	2	96.0%
Non context sensitive			
Others (GPT)	38	0	100%
Others (LLaMA)	38	0	100%

[†] : The number of invalid test case out of 50 sampled APIs.

testing on OAuth-level permission violations for email. For the remaining six host applications, we test all three permission escalations. The specific risky APIs for each host application are listed in Table VI.

a) *Effectiveness of API categorization*: We apply manual efforts to measure the effectiveness of TAI. Due to the uneven distribution of permission groups, random sampling would result in APIs belonging to the *view* or *modify* categories dominating the selection, which would not accurately reflect the performance of TAI. Following the methodology outlined in previous studies [10], [18], we randomly select around 200 APIs (20 APIs per permission group) and check the correctness of permission categorization. Our manual inspection identified only two problematic cases, achieving an accuracy of 99%. One such case involves the API `newAffineTransformBuilder()` which is categorized into the *modify* permission group. This API returns a new `AffineTransformBuilder` that assists developers in constructing an `AffineTransform`. However, this builder does not take effect immediately unless explicitly applied to a specific page like `AffineTransform.insertTo(page)`. The method `waitAllDataExecutionsCompletion(timeoutInSeconds)` also encounters a similar issue.

Specifically, some APIs may be incorrectly categorized into the modifying group even though they only require view permissions. Such misclassification can introduce false positives in our analysis, as these APIs do not actually possess modification capabilities despite being classified as modify-permission APIs.

b) *Effectiveness of test case generation* : We evaluate the performance of TAI during the test case generation phase, with details provided in Table VII. Our performance evaluation

focuses only on APIs that are not filtered out. For APIs that are filtered out, generating test cases would be meaningless, as they will not be tested.

The majority of APIs are filtered through **Pruning #1, #2**. Out of the 1,176 APIs being tested, 482 APIs require no parameter input, while the remaining require the parameters. Among the 622 APIs that require a valid parameter, 481 have a tutorial available, and we directly apply these tutorial codes for our test case generation. For these 70 APIs requiring a valid string constant, we query our attribute table to fetch the corresponding string value. Around 38 APIs require other parameters like Integer inputs, or Boolean inputs.

For benchmarking, we extracted 50 APIs from each group as shown in Table VII. For the group *unique class type* and *others* that contain fewer than 50 APIs, we sampled all of them. This results in a total of 221 APIs. We manually evaluated the validity of TAI-generated test cases by executing them and observing whether any runtime errors occurred. The precision is summarized in Table VII. Our evaluation shows that only a small proportion of the TAI-generated test cases are invalid. For around 91% APIs (require no parameters or with tutorial), TAI achieves excellent performance - 100% accuracy. We investigate reasons for invalid test cases for the remaining three categories:

Unique class type: All four invalid test cases in Table VII are introduced by the special *Enum* parameter. For example, one API `Presentation.appendSlide(predefinedLayout)` requires the parameter of class type `predefinedLayout`. After reviewing the documentation of `predefinedLayout`, we find this is a special *Enum* class defined by team workspaces. To use it, the developer must be able to provide a predefined type like `predefinedLayout = SlidesApp.PredefinedLayout.BLANK`. However, the current design of TAI cannot generate valid input for the *Enum* class, as these types are not included in the dependency relationship.

String constant: The two problematic cases in this category are caused by the implicit meaning of the string parameter. For example, the API `Spreadsheet.getAs(contentType)` requires a string parameter `contentType`, which is not included in our pre-built attribute table. Upon investigating the documentation, we found that `contentType` must follow a specific format, as shown below:

ContentType: the MIME type to convert to. For most blobs, `application/pdf` is the only valid option. For images in BMP, GIF, JPEG, or PNG format, any of `image/bmp`, `image/gif`, `image/jpeg`, or `image/png` are also valid. For a Google Docs document, `text/markdown` is also valid.

In this case, TAI fails to provide a valid input for `contentType`.

c) *Effectiveness of API testing* : Although team workspaces offer a vast array of APIs, we do not need to invoke all of them for our testing. We avoid testing scenarios that are inherently safe, such as an editor executing APIs requiring edit permissions.

Despite the large number of APIs, pruning strategies (**Pruning #1, #2**) effectively filter out 74% of them as shown in Table VI. Out of the 4,523 APIs, only 1,176 APIs would go through the risky API testing phase. The pruning strategies significantly improve the testing efficiency of *TAI* by eliminating unnecessary paths. The testing phase takes about four hours in total, with each API averaging 12 seconds to complete.

C. Large-Scale Scanning and Findings

In Table VI, the last two columns present the number of potentially problematic APIs identified by *TAI*. More specifically, *TAI* reported 54 APIs that may align with our attack scenarios. Since some APIs return null values or encrypted text that do not disclose confidential information, we conducted a manual inspection and confirmed that 41 of them pose security risks and match our criteria. For example, while the API `File.getSharingPermission()` successfully executes, it returns a NULL value and does not expose confidential data that the user cannot access. These false positives were filtered out through manual review.

Among the identified risky APIs, 17 pose E_2 risks, 21 pose E_3 risks, and none are classified as E_1 risks. The output of *TAI* shows that team workspaces are robust against **OAuth-level** attack scenarios (E_1). Team workspaces enforce strict OAuth-level permission check, ensuring installed add-ons do not exceed their authorized scopes. This robustness is expected. However, numerous permission escalations (E_2) arise due to inconsistent checking of the **resource user role** for both users and add-ons. Furthermore, add-on's ability to modify $\mathbb{C}$ without the admin's awareness (E_3) puts all invited collaborators at risk. We discuss the identified risky APIs that *TAI* detects and demonstrate the attacks that can be launched through several case studies.

a) *Hidden value leakage* : Team workspaces provide rich functionality for spreadsheets, including the ability to hide certain sheets, rows or columns. Only collaborators with *unhide* permissions can unhide and view these hidden values. However, team workspaces impose no restrictions on add-on APIs, allowing viewers without *unhide* permissions to recover hidden values via the exposed APIs. In our experiment, we concealed a *salary* column and prevented viewers from copying, downloading, or printing the spreadsheet (settings that the owner can adjust as needed). Under this setting, viewers can only view the resource content online and cannot access or recover the hidden salary column. We first tested the viewer's ability to execute `row.unhide()` API and it returned an error. However, by utilizing `row.getCell()` API, add-on installed on the viewer account can successfully fetch the hidden values even if the viewer user is forbidden from viewing the hidden values.

Even worse, team workspaces would explicitly prompt a warning notification when the owner hides a specific sheet.

Use the *view* menu to unhide sheets. All editors of this spreadsheet can view and unhide hidden sheets.

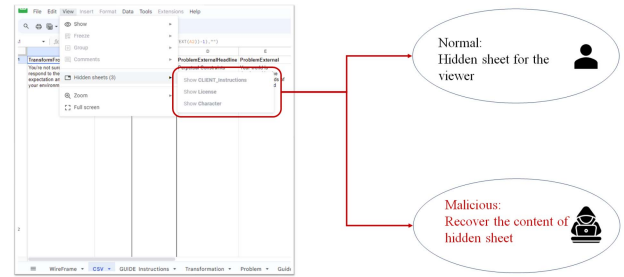


Fig. 7. Hidden sheet recovery attack.

In this case, viewers are prohibited from accessing the hidden sheets; however, by utilizing the fine-grained API `Range.getCell()`, an attacker without proper permissions can recover the entire hidden sheet as shown in Fig. 7.

b) *Protected range edition* : Additionally, we observed that spreadsheets allow the owner to set specific ranges to be protected and only editable by a specific group of people (referred to as privileged editors), rather than all editors (referred to as common editors). However, common editors are still permitted to edit the unprotected ranges. Once the protected range is *grouped*, common editors are forbidden from editing or ungrouping the whole group. However, our experiment shows that in this scenario, even though common editors are unable to either ungroup the range or edit each cell in the group, APIs exposed to add-on can still pass the execution and edit the cell value in this group. Even though the add-on fails to execute the `range.ungroup()` API.

c) *Modification of user subject* : The 21 APIs allow the add-on to modify collaborators' subject roles without the awareness of the administrator. We observe that team workspaces have already banned the use of the dangerous API [19] `transferOwnership()`. Currently, transferring ownership must be done manually by the actual user rather than be initiated by an add-on. To prevent misuse, team workspaces should also implement stricter management of the APIs that could be exploited by add-ons.

VI. DISCUSSION

A. Generalization to Other Platforms

OneDrive follows a dependency architecture that is similar in nature to Google Workspace, where APIs are organized around comparable class-based structures and dependency relationships. Therefore, the core components of *TAI*, including the dependency graph construction and test case generation, can be directly reused without modification. The main difference lies in the API specifications and documentation format [20]. To adapt *TAI* to a new platform such as OneDrive, we only need to provide the corresponding API specifications as input. *TAI* can then automatically construct the dependency graph and generate test cases based on these specifications. Furthermore, the detailed user roles in OneDrive differ slightly from those in Google Workspace. Google Workspace defines five roles: owner, editor, commenter, viewer, and unauthorized user. In

contrast, OneDrive defines five roles as well: owner, can edit, can view, cannot download, and unauthorized user. Therefore, the role distribution needs to be adjusted accordingly compared to Google Workspace. However, this remains a one-time effort and takes approximately one hour to complete. The execution environment of OneDrive follows a mechanism similar to that of Google Workspace, and we only adjust the environment configuration of the test case execution to adapt to the OneDrive environment, while keeping the rest of the code unchanged. This demonstrates that *TAI* is generalizable across different workspace platforms with similar architectural patterns, requiring only minimal adaptation at the input specification level and configuration adaptation.

Our approach can be readily extended to other collaborative platforms. However, the resulting security claims may vary depending on the underlying platform design and permission model. For example, as discussed in the Problem Definition section, the OneDrive platform does not provide APIs that allow add-ons to directly modify user roles or sharing permissions. Instead, role management is restricted to manual user-side configuration through the platform interface. Consequently, privilege-related claims that hold for Google Workspace may not necessarily apply to OneDrive.

B. Uniqueness of *TAI*

There are several challenges in training a *TAI*-like LLM to achieve the same objective. First, the LLM is constrained by its context token limit and cannot ingest the full dependency relationships that are essential for generating test cases. Second, LLMs tend to hallucinate when generating test cases from official documentation, often producing invalid parameter types that do not exist. In contrast, *TAI* generates context-aware test cases directly from the official documentation provided to developers, which helps ensure correctness. Third, API execution requires simulating developer-like behaviors, such as copying and pasting code into an editor using tools like PyAutoGUI. However, an LLM embedded in a workplace setting is typically unable to perform such interactive behaviors in a reliable manner.

C. Sharing Concerns

Team workspaces provide additional mechanisms for managing version history of non-native resources such as PDFs, images, and videos. It is important to note that native resources like Google Docs, Sheets, and Slides employ a different type of version history management. For non-native resources, users have the option to replace an existing file. For example, when a user uploads a PDF file to their workspace and a file with the same name (referred to as the old version) already exists, the user can choose to replace the old version with the new one.

While the replacement option offers convenience, it poses a potential vulnerability: the sharing attributes of the old version are also copied to the new version. This introduces a security risk, especially in relation to malicious add-ons. These

add-ons can exploit the replace option by strategically creating a *placeholder* file with crafted names, such as *salary-report-2024.pdf*, and storing it in the victim's workspace. The add-ons can then add the attacker as a collaborator for this resource. Later, when the user uploads the actual salary file and chooses to replace the placeholder file, the attacker gains automatic access to the salary report, even if the add-on is no longer installed on the user's workspace. This scenario highlights significant security implications that must be addressed to prevent unauthorized access facilitated by the replace option. This attack is practical and has been demonstrated in previous studies [21].

D. Countermeasures

In this section, we suggest countermeasures to help team workspaces strengthen their security against vulnerabilities.

Fine-grained permission management for add-on: Currently, team workspaces offer more stringent and fine-grained permission management for actual users. For example, spreadsheets allow for fine-grained permissions down to the row level. However, the inconsistency between strict user-level access control and the more coarse-grained add-on-level access control introduces numerous security vulnerabilities, as we have previously discussed. Team workspaces must enforce consistent and strict access control measures for add-ons to protect sensitive resources.

Strict management of sharing privilege: Team workspaces maintain strict control over resource content, but their management of resource-sharing permissions is very loose and flat. We recommend that team workspaces distribute only resource content-related permissions to collaborators, rather than resource management permissions. Furthermore, the sharing configuration should not be abused by add-ons. Only allowing manual operation from the user side, like the discussed API `transferOwnership()` would make the resource more secure.

E. Limitations

With all the findings discovered in our paper, our work is still preliminary due to the following reasons. First, our work primarily focuses on the problem of the three permission escalations. Thus, *TAI* is unable to detect unseen security vulnerabilities that do not meet our criteria. Second, for the rich context parameter of API invocation, we heavily rely on tutorial code snippets provided by team workspaces. If no tutorial is available, we randomly choose a valid parameter value based on the dependency relationship. In the future, we can employ LLM with the dependency relationship and feed API description as a prompt to generate context-sensitive parameters that can work well even without tutorials.

VII. RELATED WORK

A. Team Workspace Permission Analysis

Previous studies mainly focus on the security and privacy of team workspaces based on manual analysis. Wan et al.

investigate the interaction between different user roles and resources by manual analysis. Their analysis reveals many security violations that lead to permission escalation. Our work builds on their findings and is the first to systematically scan for problematic APIs that may lead to permission escalation in team workspaces.

Besides team workspaces, message systems like Slack and Microsoft Teams also enable third-party applications to join as bots or delegators to invited channels and access the message history in team chat. Several studies [10], [11], [22], [23], [24] reveal that third-party apps in the chat system can hijack other apps, steal messages from channels that they are not invited to, or perform malicious actions like merging GitHub pull requests without user's awareness. These security issues can lead to severe impacts considering the sensitive message history and resources in the team chat system.

B. Third-Party App Security

The security of third-party applications has been widely studied across different domains like the **Web Browser**, **Android**, **Internet of Things (IoT)** and **Message** systems.

a) *Browser side* : AuthScan [25] identifies design flaws in the single-sign-on (SSO) web authentication protocol that result in seven security vulnerabilities and impacted millions of users. SSOScan [26] extends this work by developing an automated vulnerability checker for applications using single-sign-on, revealing that many top-ranked websites are susceptible to SSO vulnerabilities. Some studies [27], [28] have demonstrated that browser extensions can be malicious and have created automated tools to detect malicious extensions. Other studies [29], [30] have developed checkers for privacy policy [31] declarations, identifying numerous violations.

b) *Android side* : Wang et al. [32] conduct the first systematic study on mobile cross-origin risks and demonstrate that the absence of origin-based protection allows numerous attacks. Yang et al. [33] and Zhang et al. [34] attempt to identify cross-mini-app security vulnerabilities in emerging mini-apps installed on WeChat or Snapchat.

c) *IoT side* : Recently, researchers [35], [36], [37] have expanded the scope of this topic to encompass IoT platforms such as IFTTT and voice assistant devices. Bastys et al. [38] conduct the first analysis of IFTTT flows and discover numerous security vulnerabilities leading to confidential data leakage of third party applications. Mahadewa et al. [39] examine cross-app flows in IFTTT and identify many violations of privacy regulations. Additionally, SkillScanner [40] scrutinizes both the front-end and back-end code of Amazon Alexa skills, uncovering numerous privacy violations such as excessive data requests.

d) *Message system side* : Message systems like Slack and Microsoft Teams also enable third-party applications to join as bots or delegators to invited channels and access the message history in team chat. Several studies [10], [11], [22] reveal that third-party apps in the chat system can hijack other apps, steal messages from channels they are not invited to, or perform malicious actions like merging GitHub pull requests without the user's awareness. These security issues can lead to severe

impacts, considering the sensitive message history in the team chat system.

In contrast to previous works, the distinctive nature of team workspaces introduces new security vulnerabilities. For example, team workspaces enable multi-user collaboration under various permission levels on the same resource, which introduces fresh access control risks such as permission escalation.

C. API Analysis and Testing

Unlike the commonly used REST APIs [10] for web applications, which are flat, APIs in team workspaces are more complex, featuring a well-structured design with interdependencies. Static analysis of such structured APIs [41] presents a challenge. Recent work such as IAcFinder [8] focuses on detecting access control inconsistencies between native (aka C++) and Java contexts when accessing users' confidential data stored on mobile devices. IAcFinder and TAI share similar goals and face similar challenges. They heavily depend on call graph analysis of Android libraries to generate test cases and identify security violations. In contrast to existing work, we must address API dependencies and parameter generation while considering the unique aspects of team workspaces. To achieve this, we implemented TAI to manage dependencies and efficiently prune branches as needed, expediting our testing process.

VIII. CONCLUSION

We conduct a comprehensive study of the access control system within team workspaces, focusing on resource management. We outline the two-level permission management framework for the add-on to access sensitive resources and construct attack scenarios that lead to permission escalation, providing formal representations of these scenarios.

Utilizing the permission model established by team workspaces, we developed and implemented an automated API testing tool called TAI, which identifies APIs that diverge from the specified permission management protocols. TAI successfully identified 41 high-risk APIs and provided detailed analyses. In response to the identified security vulnerabilities, we offer countermeasures to help team workspaces mitigate these risks. We hope our analysis provides valuable insights into the security analysis of team workspaces and encourages further research in this field. Our findings serve as a wake-up call for both team workspaces and add-on developers.

REFERENCES

- [1] Google workspace marketplace, 2024. [Online]. Available: https://en.wikipedia.org/wiki/Google_Workspace_Marketplace
- [2] Apps and services, 2024. [Online]. Available: <https://www.microsoft.com/en-au/microsoft-365/products-apps-services>
- [3] Google workspace user stats (2023), 2024. [Online]. Available: <https://explodingtopics.com/blog/google-workspace-stats>
- [4] L. Wan, K. Wang, H. Wang, and G. Bai, "Is it safe to share your files? An empirical security analysis of Google workspace," in *Proc. ACM Web Conf.*, 2024, pp. 1892–1901.
- [5] D. G. Balash, X. Wu, M. Grant, I. Reyes, and A. J. Aviv, "Security and privacy perceptions of third-party application access for Google accounts," in *Proc. 31st USENIX Secur. Symp.*, 2022, pp. 3397–3414. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity22/presentation/balash>

- [6] T. Bui, S. Rao, M. Antikainen, and T. Aura, "XSS vulnerabilities in cloud-application add-ons," in *Proc. 15th ACM Asia Conf. Comput. Commun. Secur.*, 2020, pp. 610–621.
- [7] M. H. Meng et al., "Post-GDPR threat hunting on android phones: Dissecting OS-level safeguards of user-unresettable identifiers," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2023, pp. 1–18.
- [8] H. Zhou, H. Wang, X. Luo, T. Chen, Y. Zhou, and T. Wang, "Uncovering cross-context inconsistent access control enforcement in android," in *Proc. 2022 Netw. Distrib. Syst. Secur. Symp.*, 2022, pp. 1–18.
- [9] Add-ons types, 2024. [Online]. Available: <https://developers.google.com/apps-script/reference/>
- [10] M. Zha et al., "Hazard integrated: Understanding the security risks of app extensions on team chat systems," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2022, pp. 24–28.
- [11] Y. Chen, Y. Gao, N. Ceccio, R. Chatterjee, K. Fawaz, and E. Fernandes, "Experimental security analysis of the app model in business collaboration platforms," in *Proc. 31st USENIX Secur. Symp.*, 2022, pp. 2011–2028.
- [12] L. Wan, K. Wang, K. T. Mahadewa, H. Wang, and G. Bai, "Don't bite off more than you can chew: Investigating excessive permission requests in trigger-action integrations," in *Proc. ACM Web Conf.*, 2024, pp. 3106–3116.
- [13] Selenium automates browsers. That's it!, 2024. [Online]. Available: <https://www.selenium.dev/>
- [14] Models: Learn about the diverse set of models that power the OpenAI API, 2024. [Online]. Available: <https://platform.openai.com/docs/models>
- [15] Build Google workspace add-ons, 2024. [Online]. Available: <https://developers.google.com/apps-script/add-ons/how-tos/building-workspace-addons>
- [16] Pyautogui, 2025. [Online]. Available: <https://pypi.org/project/PyAutoGUI/>
- [17] H. Harkous and K. Aberer, "If you can't beat them, join them," A usability approach to interdependent privacy in cloud apps," in *Proc. 7th ACM Conf. Data Appl. Secur. Privacy*, 2017, pp. 127–138.
- [18] C. Wang, R. Ko, Y. Zhang, Y. Yang, and Z. Lin, "TaintMini: Detecting flow of sensitive data in mini-programs with static taint analysis," in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng.*, 2023, pp. 932–944.
- [19] API: Consent is required to transfer ownership of a file to another user [error=403], 2024. [Online]. Available: <https://issuetracker.google.com/issues/228791253>
- [20] Office scripts JavaScript API reference, 2026. [Online]. Available: <https://learn.microsoft.com/en-us/javascript/api/office-scripts/officescript?view=office-scripts>
- [21] N. Nikiforakis, S. Van Acker, W. Meert, L. Desmet, F. Piessens, and W. Joosen, "Bitsquatting: Exploiting bit-flips for fun, or profit?," in *Proc. 22nd Int. Conf. World Wide Web*, 2013, pp. 989–998.
- [22] P. Rösler, C. Mainka, and J. Schwenk, "More is less: On the end-to-end security of group chats in signal, WhatsApp, and Threema," in *Proc. 2018 IEEE Eur. Symp. Secur. Privacy*, 2018, pp. 415–429.
- [23] C. Yan, B. Guan, Y. Li, M. H. Meng, L. Wan, and G. Bai, "Understanding and detecting file knowledge leakage in GPT app ecosystem," in *Proc. ACM Web Conf.*, 2025, pp. 3831–3839.
- [24] C. Yan, R. Ren, M. H. Meng, L. Wan, T. Y. Ooi, and G. Bai, "Exploring ChatGPT app ecosystem: Distribution, deployment and security," in *Proc. 39th IEEE/ACM Int. Conf. Automated Softw. Eng.*, 2024, pp. 1370–1382.
- [25] G. Bai et al., "AUTHSCAN: Automatic extraction of web authentication protocols from implementations," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2013, pp. 1–20.
- [26] Y. Zhou and D. Evans, "SSOScan: Automated testing of web applications for single sign-on vulnerabilities," in *Proc. 23rd USENIX Secur. Symp.*, 2014, pp. 495–510.
- [27] N. Jagpal et al., "Trends and lessons from three years fighting malicious extensions," in *Proc. 24th USENIX Secur. Symp.*, 2015, pp. 579–593.
- [28] N. Pantelaios, N. Nikiforakis, and A. Kapravelos, "You've changed: Detecting malicious browser extensions through their update deltas," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2020, pp. 477–491.
- [29] D. Bui, B. Tang, and K. G. Shin, "Detection of inconsistencies in privacy practices of browser extensions," in *Proc. IEEE Symp. Secur. Privacy*, 2023, pp. 2780–2798.
- [30] Y. Ling, K. Wang, G. Bai, H. Wang, and J. S. Dong, "Are they toeing the line? Diagnosing privacy compliance violations among browser extensions," in *Proc. 37th IEEE/ACM Int. Conf. Automated Softw. Eng.*, 2022, pp. 1–12.
- [31] C. Yan, F. Xie, M. H. Meng, Y. Zhang, and G. Bai, "On the quality of privacy policy documents of virtual personal assistant applications," in *Proc. Privacy Enhancing Technol.*, 2024, pp. 478–493.
- [32] R. Wang, L. Xing, X. Wang, and S. Chen, "Unauthorized origin crossing on mobile platforms: Threats and mitigation," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2013, pp. 635–646.
- [33] Y. Yang, Y. Zhang, and Z. Lin, "Cross miniapp request forgery: Root causes, attacks, and vulnerability detection," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2022, pp. 3079–3092.
- [34] L. Zhang et al., "Identity confusion in WebView-based mobile app-in-app ecosystems," in *Proc. 31st USENIX Secur. Symp.*, 2022, pp. 1597–1613.
- [35] M. Surbatovich, J. Aljuraidan, L. Bauer, A. Das, and L. Jia, "Some recipes can do more than spoil your appetite: Analyzing the security and privacy risks of IFTTT recipes," in *Proc. 26th Int. Conf. World Wide Web*, 2017, pp. 1501–1510.
- [36] Y. Chen, M. Alhanahnah, A. Sabelfeld, R. Chatterjee, and E. Fernandes, "Practical data access minimization in trigger-action platforms," in *Proc. 31st USENIX Secur. Symp.*, 2022, pp. 2929–2945.
- [37] M. M. Ahmadpanah, D. Hedin, and A. Sabelfeld, "LazyTAP: On-demand data minimization for trigger-action applications," in *Proc. IEEE Symp. Secur. Privacy*, 2023, pp. 3079–3097.
- [38] I. Bastys, M. Balliu, and A. Sabelfeld, "If this then what? Controlling flows in IoT apps," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2018, pp. 1102–1119.
- [39] K. Mahadewa et al., "Identifying privacy weaknesses from multi-party trigger-action integration platforms," in *Proc. 30th ACM SIGSOFT Int. Symp. Softw. Testing Anal.*, 2021, pp. 2–15.
- [40] S. Liao, L. Cheng, H. Cai, L. Guo, and H. Hu, "SkillScanner: Detecting policy-violating voice applications through static analysis at the development phase," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2023, pp. 2321–2335.
- [41] C. Yan, M. H. Meng, F. Xie, and G. Bai, "Investigating documented privacy changes in android OS," *Proc. ACM Softw. Eng.*, vol. 1, no. FSE, pp. 2701–2724, 2024.



Liuhuo Wan received the master's degree from the National University of Singapore, and the PhD degree from the University of Queensland, where she focused on cybersecurity. Her research interests include software security, code analysis, and network security.



Chuan Yan received the master's and PhD degrees from the University of Queensland. He is a post-doctoral fellow with the City University of Hong Kong. His research focuses on detecting security and privacy issues in third-party apps integrated with large language models, based on privacy documentation.



Zihan Wang is currently a research fellow with the University of Queensland, Australia. He was previously a visiting scientist with CSIRO, Australia. His research interests include AI security and privacy, systems and software security, and formal verification. He received the 2024 Google PhD fellowship in security, privacy, and abuse prevention.



Haoyu Wang is currently a full professor with the School of Cyber Science and Engineering, Huazhong University of Science and Technology (HUST). He is leading the SECURITY PRIDE Research Group. His research covers a wide range of topics in software analysis, privacy and security, eCrime, Internet/System measurement, and AI security. He has been awarded three best/distinguished paper awards, including WWW 2020 Best Student Paper Award (the first award from China), and ACM OOPSLA 2020 Distinguished Paper Award.



Mark Huasong Meng received the bachelor of engineering (hons.) degree from Nanyang Technological University (NTU), Singapore, and the master's and PhD degrees from the National University of Singapore (NUS). He is an assistant professor with the School of Computer Science, University College Dublin (UCD), Ireland. His research interests include trustworthy AI, software engineering, and mobile security.



Guangdong Bai (Member, IEEE) received the BS and MS degrees in computer science from Peking University, Beijing, China, in 2008 and 2011, respectively, and the PhD degree in computer science from the National University of Singapore, Singapore, in 2015. He is currently an associate professor with the Department of Computer Science, City University of Hong Kong, Hong Kong. His research interests include cyber security, software engineering, and machine learning.



Kailong Wang received the bachelor's (first class honours) degree from Nanyang Technological University (NTU), and the PhD degree from the National University of Singapore (NUS). He is an associate professor with the Huazhong University of Science and Technology (HUST), China. He is broadly interested in trustworthy AI, secure and private software engineering. His current research focuses on the safety and ethics assessment as well as interpretability and explainability of large language model.



Jin Song Dong received the PhD degree from the University of Queensland, Australia. He is a professor with the School of Computing, National University of Singapore. His research interests include software engineering, program analysis, formal verification, and model checking. He has been on the editorial board of the *ACM Transaction on Software Engineering and Methodology*, *Formal Aspects of Computing*, and *Innovations in Systems and Software Engineering*, *A NASA Journal*.